

Which Part of the Skill Does the Work?

A Component-Level Analysis of Procedural Knowledge Injection in LLM-Based Malware Traffic Analysis

Yaakulya Sabbani

Security Engineer (R&D)

Center for Cybersecurity, New York University

Brooklyn, New York 11201, USA

yaakulya.sabbani@nyu.edu

Abstract—The research aims at determining whether it is possible to transform a general LLM agent into a domain-specific agent by incorporating Agent Skills (specific procedural knowledge placed into the model context while performing inferences) and which element of the skill determines the strongest impact on the process. For this purpose, the model, task, and toolkit were kept constant, while only the skill was varied in nine experimental conditions applied to nine actual malware infection packet captures with answer keys verified packet by packet. These experimental conditions included: (1) no skill, (2) four widely used community skills, (3) our IOC-forensics skill, and (4) three shortened versions of our skill in which one of the following components was removed: the workflow, the `tshark` command recipes, or the output schema. The accuracy was constant. Victims were almost always recognized correctly in all conditions, IOC recall did not differ across arms ($p=0.08$), and fabrication of indicators was rare, with no difference between any pair of arms. What changed was effort. Our skill consumed 30% fewer `tshark` commands compared to the base agent (paired $p<0.001$, $d_z=-1.71$; this difference persists after applying Holm-Bonferroni correction and the Wilcoxon test, reaching even 35% on a second model) and 22% fewer commands than the community skills. Output tokens (11%) and wall-clock time (13%) were reduced too, but not significantly. Our skill provides a more efficient tool-use pattern without increasing the quality of answers. From three components of the skill, only the recipes increase the number of commands in case of their deletion, for both models; the results of the workflow and schema components are reversed across models. An additional fact is that 4 of the 9 published answer keys had wrong primary network indicators, and we fixed this error through comparing the packets.

Index Terms—network forensics, large language models, agent skills, packet analysis, indicators of compromise, incident response, `tshark`

I. INTRODUCTION

Triage of the packet capture analysis process is labor-intensive and requires both time and expert knowledge. An analyst needs to identify an infected host, determine the malware family, and develop a set of indicators of compromise (IOCs) to include in the incident report. Large language models (LLMs) are considered as one of the ways to simplify the process. Contemporary systems use protocol analyzers automatically, and it has been shown that packet captures include enough evidence for an agent to analyze [1]–[3]. In addition, the same period of time saw the development of the

concept of *Agent Skills* [4], [5]: the packaging of procedural knowledge, including command sequences and instructions, required for a generalist model to be a specialist in a particular domain. Cross-domain validation showed considerable performance improvements thanks to acquired skills, with cybersecurity being one of those domains [6].

However, two questions remained unclear. First, whether a skill makes the `tshark`-driven agent a proficient forensic analyst when evaluated against verified ground truth data and not its own reports. Second, an even more fundamental question concerns the skill itself: *what inside a skill does the work?* While a skill consists of procedural workflow, tool recipes, and output formatting, previous research considered the whole combination as a unity. It might be the case that only one component brings most of the value, in which case skill authors spend effort in the wrong place.

We consider the two questions above by framing skill as the independent variable. Holding the model, task, and toolbox constant, we vary only the skill injection across nine experimental arms (Section III). In three arms, a purpose-built skill loses one specific component to provide an apples-to-apples comparison with the complete skill. Similar studies ablate skills in offense-oriented CTF settings [7] and in data science pipelines [8]; both lines of work find no benefit in terms of accuracy. Skill impact in general tasks varies from task to task and largely cancels out when aggregated [9]. Cost-conscious analysis of security agents is a separate, burgeoning stream of literature [10]. We apply the same ablation method to investigate the impact of skills in *defense-oriented network and malware traffic forensics*; in this setup, we believe ourselves to be the first to compare a purpose-built skill against a *population* of widely used community skills. The positive outcome is purposely limited to a very specific scenario. When the model, task, and toolbox are held constant, a custom-built, domain-specific skill significantly *decreases the number of calls made by the agent to its tools* on two models, compared with no skill and with the community population, all while keeping the accuracy unchanged (Fig. 1). Both the cost metric for tool calls and the goal of minimizing them for a given level of accuracy are not new contributions [10], [11]. What we show is that domain skills injected via prompts result in the

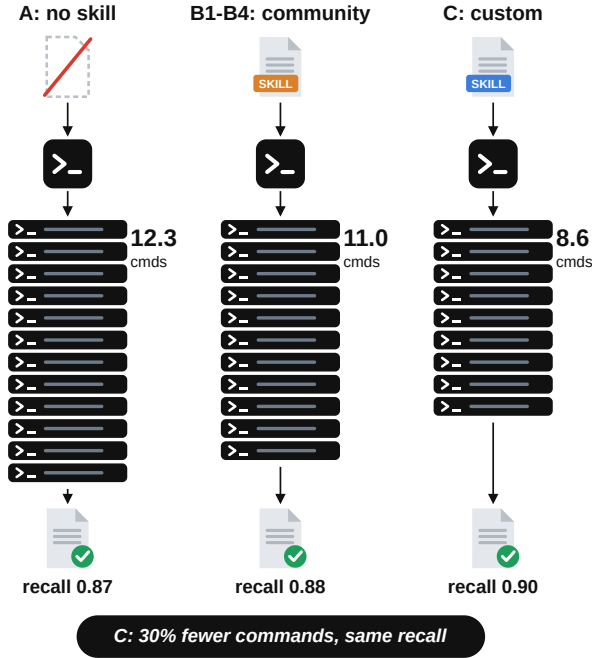


Fig. 1. Same agent, three skill conditions (primary model, Sonnet). Each chip is one *tshark* command, drawn to the mean count per case; every condition reaches a report with near-identical IOC recall.

decrease as a consequence, while none of the four community skills produces a significant reduction on the primary model.

Contributions.

- A controlled nine-arm experiment design which isolates the contribution of a skill from the contributions of its component parts using an LLM-agent network-forensics setup (Section III).
- A validated nine-capture forensic benchmark with typed and evidence-checked ground truth IOCs for RAT, stealer, and loader classes. Curation reveals a previously undiscovered erroneous primary network indicator (C2 or victim IP and one victim MAC) in published answer keys, which we corrected (Section IV).
- Investigation of the influence of skill and component effects on the IOC recall, attribution, hallucination, efficiency, and cost of the skill (Section V). The skill results in a statistically significant reduction in number of *tool calls*, surviving multiple comparison adjustment and consistent across two models without any decrease in accuracy and with indistinguishable faithfulness; we found that the skill effect is localized to command recipes through the ablation study. We further introduce the *skill-issue effect*, where a specialized skill may constrain the exploration a generalist agent would perform. This effect was discovered during piloting and addressed during the experiment design phase (Section VI).
- Open harness, data set, and skills corpus to support reproducibility.

II. RELATED WORK

A. LLM agents, reasoning, and tool use

The large language models of today are all based on transformers [12]. In-context learning [13] and instruction tuning [14] allow one model to be steered via its prompt input. Chain-of-thought prompting [15] and self-consistency [16] elicit reasoning, which can be scaled with respect to model size [17]. Agency is achieved through alternating between reasoning and action: ReAct [18] mixes reasoning and calling of tools, Toolformer [19] and ToolLLM [20] condition the model to call tools externally, and Reflexion [21] brings self-critique into the mix. Evaluation has moved beyond measuring just the accuracy of the final answer towards measuring the reasoning process itself [22] and towards software tasks like SWE-bench [23]. When looking at the problem in such terms, skills are ways of conditioning a tool-using loop, and the key question is which parts of that conditioning are decisive [24].

B. Agent Skills and procedural knowledge injection

Agent Skills instantiate procedural knowledge (instructions, command recipes, output contracts) into the generalist model at inference time [4], [5], [25]. SkillsBench benchmarks skills on a large suite of tasks and finds significant improvement, including in cybersecurity [6], whereas recent research conceives skills as an institutional knowledge primitive [26]. In both cases, a skill is considered a whole, whereas we disassemble it. There is only a handful of contemporaneous works which conduct either ablations or stress tests for agent skills, and they reach rather discouraging results. Chacko et al. [7] re-analyze a 180-run offensive CTF experiment from the point of view of skill ablation and find no reliable increase in accuracy. Huang [8] deconstructs the structure of auto-generated data-science skills and finds that the procedural overhead results in increased token consumption without improving task performance. Wang et al. [9] show, in a variety of tasks, that a lot of skills do not help and may even harm. Our paper differs from them in three aspects. Firstly, the domain is defense-oriented network/malware traffic forensics, not CTF, data science, or generic tasks. In addition, the baseline consists of a population of four widely adopted community skills rather than a single skill or an auto-generated one, thus avoiding selection bias in the comparison. The reported cost pertains to the agent's *behavioral* cost and is captured by the number of tool calls made, instead of the payload cost of skill content injection quantified via the data-science ablation (a 4.5x increase in input tokens without any gains in accuracy). The axis of tool calls is not a novel one: cost-aware evaluation of security agents is an active area of research, and Kassianik et al. [10] argue that disciplined tool use is what matters for the success of defensive-SOC agents. Reduced tool calls at a given accuracy is not an entirely novel goal either; there is a whole efficiency literature that aims to do so, and the Skill-to-LoRA [11] technique distills skills into adapters precisely for the purpose of reducing the token cost per step. What our claims pertain to is something more modest: in a fixed model-

task-toolbox setting, a manually designed, domain-matched *prompt-injected* skill decreases tool calls as a by-product (30% fewer calls vs. no skill, replicated on two models), while none of the four community skills significantly reduces tool calls on our primary model. This is consistent with the data-science results regarding the token overhead: the two costs are distinct and can co-exist, since skill injection still brings input tokens. We show that the behavioral cost of a skill can decrease while its payload cost increases, depending on the quality of the skill construction and its relevance to the domain. Structured output constraints are known to reduce tool calling in open-weight models [27], which is consistent with the observation that removing an inflexible output schema does not harm performance.

C. LLMs for cybersecurity and the SOC

Today, LLM agents have applications in both offensive and defensive security realms [28], [29]. Penetration testing surveys highlight autonomous frameworks [30], and critical analyses find hallucination, especially flag hallucination, widespread across them [31]. The findings are an impetus for evidence-based evaluation similar to what we propose in this paper. Semi-autonomous solutions retain a human-in-the-loop feature [32]. In defense, there are knowledge-grounded agents [33], retrieval-enabled solutions for novel threats [34], and alert triage to alleviate SOC overload problems [35]. The closest work to our ablation study is Jang et al. [36], who develop a local LLM pipeline to analyze security incidents and perform ablations of the pipeline components, namely removal of the orchestration layer and deactivation of the verification module, the latter raising hallucination. We conduct ablations of the skill components, keeping the agent architecture intact. CyberSecEval [37] and CyberSOCEval [38] evaluate the security capabilities of agents but do not investigate the skill mechanism.

D. LLM agents for network forensics

CyberSleuth [1] is closest to our study: an autonomous blue-team agent with a *tshark* sub-agent specialized in forensic analysis of web attacks. Holmes [2] orchestrates tools for auditable DDoS attack investigation. SIABench [3] evaluates the security incident analysis capabilities of LLM agents, including network forensics, using the scenarios created by blue-team training platforms like CyberDefenders and a real-world test that includes the challenges offered by the Malware-Traffic-Analysis.net website. TrafficLLM [39] adapts LLMs to traffic representation, and GRIDAI [40] and IoT-traffic agents [41] are aimed at rule generation and interpretation. Overall, these studies show that agents are capable of extracting forensic evidence from packet captures. No work isolates and decomposes the skill mechanism, which is the main subject of our paper.

E. Machine learning for network intrusion detection

Traditionally, network defense was implemented via signature engines (Suricata [42], Zeek [43]), and nowadays machine

learning classifiers are being developed based on labeled flow data [44], [45]. The surveys cover the field of deep learning IDS and dataset trends [45], and controlled experiments on CIC-IDS2017 [46] and imbalanced datasets [47] reveal high in-distribution accuracy. They classify the traffic into attack and benign classes. Our agents give different outputs, namely a human-readable incident report with grounded indicators, mapped to the adversary’s behavior [48].

F. Hallucination and faithfulness

Hallucination, which is the claim by a model of content that lacks support from the source, is one of the main failure modes of LLMs [49], [50]. In the context of forensics, this issue becomes especially significant because even a single fabricated C2 indicator costs a lot of analysis effort or, alternatively, may lead to an improper response. Therefore, fabricated indicators become a first-class metric in our evaluation and are detected by retrieval-based grounding [51] against the packet-capture data.

III. METHOD

A. The skill as independent variable

In the described scenario, the agent runtime (Section III-C), the task prompt, and the tool interface are held constant; the only parameter that changes is the injected skill. Table I and Fig. 2(2) list the nine experimental conditions. Arm A is the control. To make sure that the “community skill” comparison does not rest on one chosen skill (and to avoid cherry-picking), we test a *population* of four independent community skills (B1–B4). In addition, to confirm that the skills are used by practitioners (and are not obscure or idiosyncratic), all four skills are taken from the two most-starred public Claude-skill repositories (around 26k and 30k GitHub stars, respectively, at the time of writing). B1–B3 were taken from the first repository and B4 from the second one; hence two different maintainers are involved. B2 is specifically designed for malware traffic and is therefore the strongest community baseline for the current task. Arm C is the custom skill designed for this experiment. Arms D1–D3 correspond to arm C with precisely one component removed; these arms are created programmatically by removing a single marked section and differ from C by one component only.

The custom skill (arm C) consists of three separate components: (i) a *procedural workflow* describing the investigative process (evidence inventory, capture scoping, victim identification, traffic shape determination, family attribution, and validation of each indicator); (ii) *tshark recipes* corresponding to the described procedures; and (iii) a strict output *schema* requiring a typed, evidence-annotated IOC set. Each component has its own clearly identifiable section, and an ablation involves removal of one section verbatim.

B. Task and scoring

Every experimental condition comes with an identical neutral prompt: analyze the evidence directory (consisting of a packet capture, possibly augmented with exported IDS alerts)

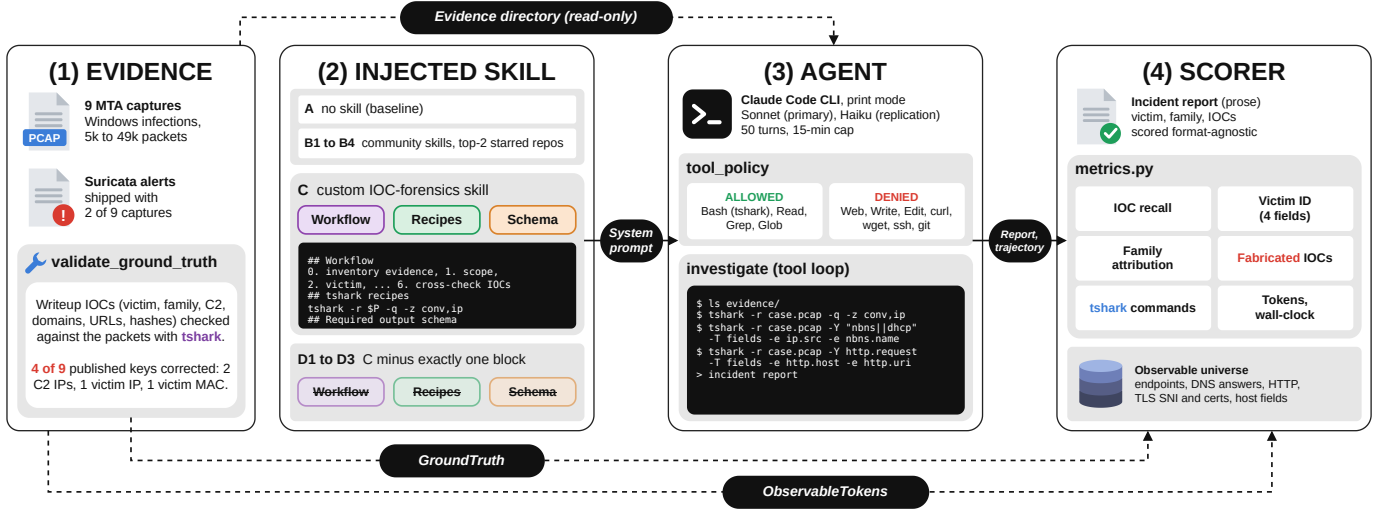


Fig. 2. Evaluation architecture. Only the injected skill (2) varies across the nine arms; the evidence (1), the agent and its read-only tool policy (3), and the scorer (4) are fixed. D1–D3 each delete one block of the custom skill C. Input icons in Fig. 4: Microsoft Fluent Emoji (MIT licence).

TABLE I
THE NINE EXPERIMENTAL ARMS. ONLY THE INJECTED SKILL DIFFERS.
B1–B4 ARE A POPULATION OF INDEPENDENT, PUBLICLY-AVAILABLE
COMMUNITY SKILLS.

Arm	Injected skill	Components present
A	none (vanilla)	n/a
B1	community: Wireshark (repo 1)	external
B2	community: malware-traffic (repo 1)	external
B3	community: PCAP forensics (repo 1)	external
B4	community: Wireshark (repo 2)	external
C	custom IOC-forensics skill	workflow + recipes + schema
D1	C – workflow	recipes + schema
D2	C – recipes	workflow + schema
D3	C – schema	workflow + recipes

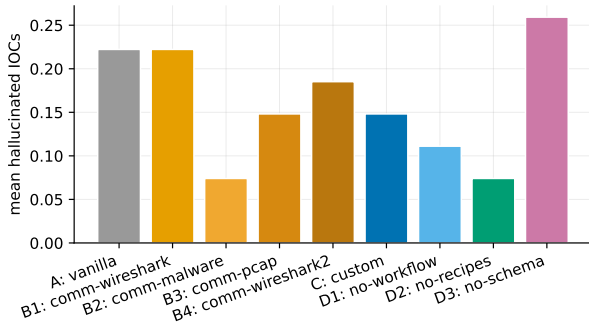


Fig. 3. Mean fabricated indicators per case by arm. A fabricated indicator is an IOC-shaped token the agent asserts that appears in zero packets. No community skill (B1–B4) exceeds the baseline (B1 ties it at 0.22), and no difference is significant at $n=9$ (Table III).

and write an incident report describing the victim host (IPv4 address, MAC address, hostname, and user), the malware family, and all grounded IOCs. The prompt asks for a prose narrative, so the schema ablation is not a source of bias, and scoring is agnostic to format. For every run (Fig. 4),

the following measurements are taken: *IOC recall* (fraction of vetted IOCs grounded in the report out of all vetted IOCs); *victim-identification accuracy* (four canonical fields); *family attribution* (correct vs. incorrect); *hallucination count* (IOC-like entities claimed by the agent and not present in the capture, measured against a per-capture universe of every observable IP, domain, and host; this universe is constructed from packet endpoints, DNS queries *and answers*, HTTP hosts, request URIs, redirect and referer headers, TLS SNI and certificate subject/SAN fields, and NetBIOS/DHCP/Kerberos host fields, excluding local-subnet infrastructure and a documented benign-reference allowlist); *tool-call efficiency* (tshark calls); and *cost* (output tokens, wall-clock time).

C. Experimental setup

The agent executes inside the Claude Code CLI in non-interactive mode. The skill is delivered as an appended system preamble, which separates procedural-knowledge *content* from skill-retrieval mechanics. The agent has access to read-only local utilities (tshark through the shell, file reading, searching) and is forbidden from accessing any web resources, file transfers, or writes to prevent any kind of solution leakage; all trajectories are logged and audited for possible leaks. Every arm uses the same model (Claude Sonnet) and follows the same constraints of 50 turns and a 15-minute wall-clock limit per run. The model is delivered as an unpinned alias, which is a reproducibility constraint we acknowledge; exact bit-level replication requires pinning down the build. The target is three repetitions per condition, and Section VII reports the coverage reached.

IV. DATASET

The 9 real malware-infection packet captures are obtained from Malware-Traffic-Analysis.net [52]. They correspond to infections of an enterprise Windows workstation and come with the official analyst write-up. Families of

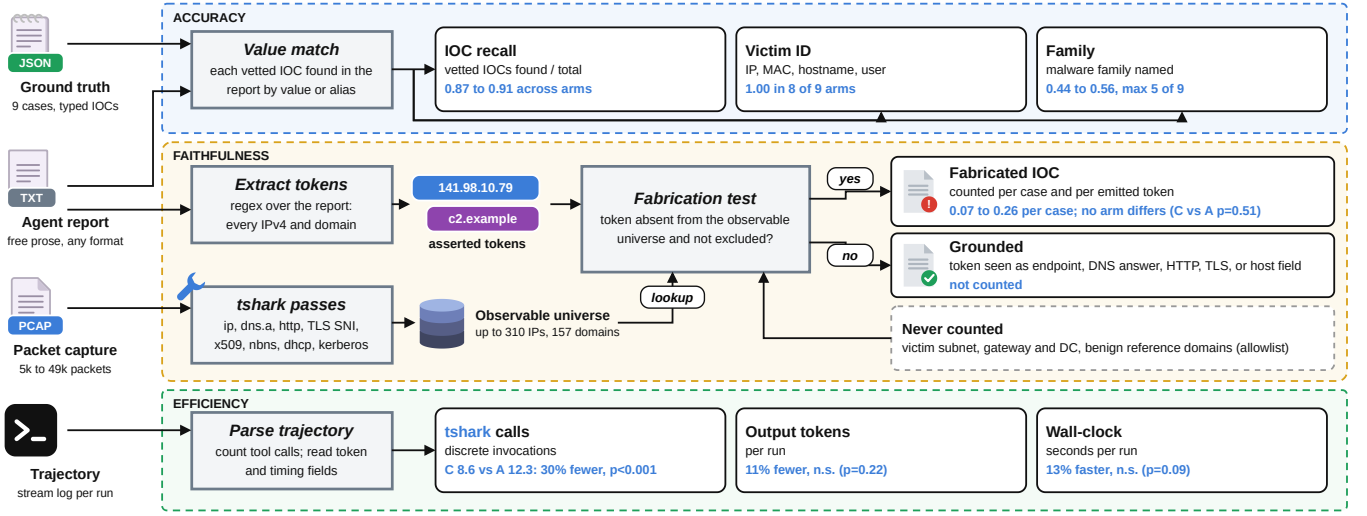


Fig. 4. How a run is scored, in three lanes. Accuracy: each vetted IOC is matched by value or alias in the prose report. Faithfulness: every IPv4 address and domain in the report is tested against an observable universe built from *tshark* field passes, with local infrastructure and benign reference domains never counted. Efficiency: tool calls, tokens, and time come from the trajectory stream. Blue lines give the measured ranges (Sonnet).

malware involved include remote-access trojans (NetSupport RAT, NetSupport Manager RAT, STRRAT), information stealers (Koi, Oski, Lumma, WarmCookie), and loaders (Bazar-Loader/TA551). Capture size ranges from 5k to 49k packets. Two captures come with exported Suricata alerts. The remaining seven cases require behavioral attribution.

Ground-truth validation. Transcription of each write-up resulted in a typed set of IOCs (victim fields, family, C2 endpoints, domains, URLs, file names and hashes). Subsequently, all network indicators were validated against their captures using *tshark*. Validation revealed a wrong primary network indicator in 4 of 9 exercises: the C2 IP address in two, the victim IP in one, and the victim MAC in one. In each case, the value printed in the key did not match anything in any packet, while the value on the actual port and session differed only slightly. For example, the STRRAT key listed 141.98.10.69, but the traffic on port 12132 was addressed to 141.98.10.79. All such values were fixed according to primary evidence and noted. After validation, the victim IPs, C2 IPs, domains, and hostnames for all 9 cases appear in their captures.

V. RESULTS

Table II reports the arm-level means per case: reps are averaged within a case, then over the 9 cases, and the 95% confidence intervals are in the released analysis output. Table III reports the paired significance tests, which separate the signal from the noise. To summarize: **the purpose-built skill provides a statistically significant decrease in the number of tool calls per investigation, with no significant change in accuracy or faithfulness.** The significant result is reported first, followed by the null.

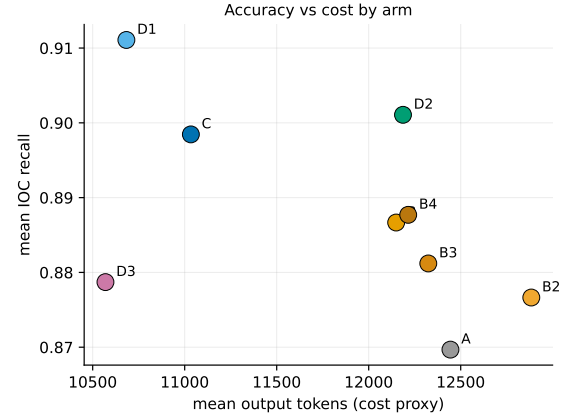


Fig. 5. Accuracy versus cost. Each point is one arm's mean.

A. The purpose-built skill significantly cuts discrete tool invocations

This represents the main finding. Compared to the untreated baseline, the custom skill (C) uses 30% fewer *tshark* invocations (8.6 vs 12.3; paired $t=-5.14$, $p<0.001$, $d_z=-1.71$), and compared to the four-skill community population, it uses 22% fewer (8.6 vs 11.0; paired $t=-7.14$, $p<0.001$, $d_z=-2.38$). The community comparison does not change much without the toolbox-mismatched skill B4 (21%, $p<0.001$; Fig. 8, right). The effect is large and holds under pairing in all 9 cases. A total of six cost comparisons were made, justifying a Holm-Bonferroni correction on this family (Table III, adjusted p in parentheses); the reduction in tool invocations remains significant ($p_{\text{adj}}=0.005$). The paired differences form a small sample that might not be normally distributed, so the non-parametric Wilcoxon signed-rank test confirms the result ($p=0.004$), with the effect size calculated using the

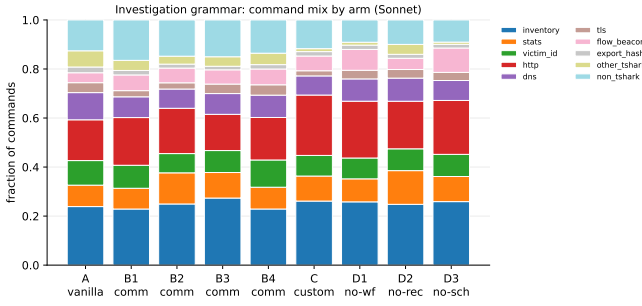


Fig. 6. Investigation grammar on the primary model (Sonnet): the share of each command category per arm. The custom skill and its ablations spend a larger share on HTTP queries and a smaller share on unclassified `tshark` and non-`tshark` commands than the baseline and the community skills.

Hedges small-sample correction ($g=-1.55$). The result does not depend on a normality assumption at $n=9$. There is also a reduction in output tokens (11%, 1.4k fewer per case) and wall-clock time (13%), but neither change is significant on this model ($p=0.22$ and $p=0.09$, respectively, and both drop out under correction). The interpretation is deliberately conservative. The changed metric is the number of *discrete* `tshark` invocations; the token and wall-clock costs a SOC actually meters do not change significantly at $n=9$. The parser treats each command as a single invocation irrespective of the number of display fields and pipes it contains, which means that some of the effect may be due to the skill batching work into fewer, richer commands rather than doing strictly less. Thus, the conclusion is a smaller *tool-invocation footprint*, rather than a proven cost or latency advantage.

One may argue against the decreased command count in a rather straightforward manner by pointing out that it simply reflects a decreased workload. However, the evidentiary yield does not support such an interpretation. Mean counts of *true* IOCs recovered do not differ significantly between arms, and among the full-skill arms (A, B1–B4, C), C has the highest count: 7.9 compared to 7.7 for the untreated baseline (out of 8.9 possible per case; two ablation arms recover marginally more). C reaches the same forensic results (the identification of the victim and the same or a higher number of grounded indicators) with fewer commands (Fig. 5) and devotes a larger fraction of those commands to HTTP queries (Fig. 6).

B. Accuracy and faithfulness: no significant difference

No arm differs from the baseline significantly with respect to any measure of accuracy (Table III, Fig. 7). Victim identification is close to perfect, with 1.00 achieved in all arms apart from the malware-specific community skill B2 (0.99, due to missing one field in a single run). IOC recall falls in a narrow band of only 0.87–0.91, and the difference between C and A is not statistically significant (the custom skill is nominally better by 0.03, $p=0.08$). A two-one-sided-tests (TOST) equivalence check at a ± 0.05 margin is marginal ($p=0.09$; equivalence is established only for margins of about ± 0.056 or more). Therefore, recall is reported as a non-significant difference

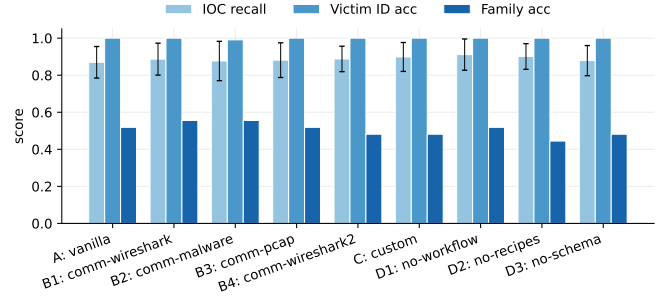


Fig. 7. IOC recall, victim-ID accuracy, and family attribution by arm.

in favor of the skill rather than a formally demonstrated equivalence. In any case, no skill *reduces* recall, and the 95% confidence intervals of all nine arms overlap.

Faithfulness shows no observable effect either. Fabrication is judged based on absence from a per-capture “universe” of observable tokens: packet endpoints, DNS query *and* answer records, HTTP hosts and request URIs, TLS SNI and certificate names, and NetBIOS/host fields (Section III). The universe deliberately gives credit for payload-based indicators that a diligent analyst would legitimately report. Under this stringent scoring, fabrication is uniformly low (0.07–0.26 indicators per case; Fig. 3), with no arm showing a significant difference from any other arm. The custom skill (0.15) does not differ from the untreated baseline (0.22, $p=0.51$), and the community skills vary between 0.07 and 0.22. Under a laxer universe, an initially favorable faithfulness effect appeared for the custom skill, but it disappears when the universe includes DNS-answer and payload fields, so we report the null. A secondary, length-normalized view agrees with the finding. As a *rate* over emitted IOC tokens, the custom skill (0.011) is lower than the untreated baseline (0.019), but with the counts null, it is not claimed as a result. The metric has an inherent limitation: it detects only gross fabrication. The universe is large (up to 310 IPs and 157 domains in the busiest capture) and domain matching is done by suffix; hence a fabricated indicator that resembles an observed one can slip through. The null result means “no arm grossly fabricates,” not that fabrication is zero. Family attribution is between 0.44 and 0.56 for all arms and varies by at most one case between arms; one capture has no labeled ground-truth family and is thus rated unattributed for all arms, with blanks scored incorrect.

As a result, both accuracy and faithfulness are nulls: no injected skill moves recall, victim identification, or fabrication. Skills differ in tool-call cost. The custom skill calls significantly fewer `tshark` commands than the community population under *both* models: 8.6 vs 11.0 on Sonnet (paired $p<0.001$) and 20.1 vs 27.5 on Haiku ($p<0.001$). This difference from real, widely used community skills is the result that holds on both models.

TABLE II

PER-ARM RESULTS ON THE PRIMARY MODEL (SONNET), CASE-LEVEL MEAN OVER 9 CASES. B1–B4 FORM THE COMMUNITY-SKILL POPULATION. RECALL, VICTIM, AND FAMILY ARE FRACTIONS; HALLUCINATION IS FABRICATED IOCS PER CASE (LOWER IS BETTER); TSHARK IS THE MEAN COMMAND COUNT; TOKENS ARE MEAN OUTPUT TOKENS. AMONG THE FULL SKILLS (A–C), THE CUSTOM SKILL (C) ISSUES THE FEWEST TSHARK COMMANDS WITH NO SIGNIFICANT RECALL DIFFERENCE, AND FABRICATION IS LOW AND STATISTICALLY INDISTINGUISHABLE ACROSS ALL ARMS.

Arm	Recall	Victim	Family	Halluc.↓	tshark↓	Tok↓
A: vanilla	0.870	1.000	0.52	0.22	12.3	12444
B1: comm-wireshark	0.887	1.000	0.56	0.22	11.1	12149
B2: comm-malware	0.877	0.991	0.56	0.07	11.1	12884
B3: comm-pcap	0.881	1.000	0.52	0.15	10.3	12324
B4: comm-wireshark2	0.888	1.000	0.48	0.19	11.7	12215
C: custom	0.898	1.000	0.48	0.15	8.6	11034
D1: no-workflow	0.911	1.000	0.52	0.11	9.1	10683
D2: no-recipes	0.901	1.000	0.44	0.07	9.9	12186
D3: no-schema	0.879	1.000	0.48	0.26	9.3	10569

TABLE III

PAIRED t -TESTS ACROSS THE 9 CASES (Δ = ARM MINUS COMPARATOR; d_z = PAIRED EFFECT SIZE; THE PARENTHEZIZED p AND THE SIGNIFICANCE STARS ARE HOLM-ADJUSTED OVER THE SIX COST CONTRASTS). ONLY THE TOOL-CALL REDUCTION (C–A) IS SIGNIFICANT; ITS EFFECT IS LARGE AND SURVIVES CORRECTION. THE TOKEN AND WALL-CLOCK REDUCTIONS POINT THE SAME WAY WITHOUT REACHING SIGNIFICANCE, AND NO ACCURACY CONTRAST IS SIGNIFICANT. *** $p < .001$, ** $p < .01$, * $p < .05$.

Metric	Contrast	Δ	t	p	d_z	sig
<i>Efficiency / cost</i>						
tshark calls	C–A	-3.74	-5.14	<0.001 (0.005)	-1.71	**
tshark calls	B1–A	-1.26	-1.30	0.230 (0.899)	-0.43	ns
output tokens	C–A	-1410.67	-1.32	0.225 (0.899)	-0.44	ns
output tokens	B1–A	-294.96	-0.22	0.831 (1.000)	-0.07	ns
wall time s	C–A	-25.37	-1.91	0.093 (0.465)	-0.64	ns
wall time s	B1–A	-4.58	-0.24	0.817 (1.000)	-0.08	ns
<i>Accuracy / faithfulness</i>						
recall	C–A	+0.03	2.00	0.081	+0.67	ns
hallucination count	C–A	-0.07	-0.69	0.512	-0.23	ns
fam	C–A	-0.04	-1.00	0.347	-0.33	ns

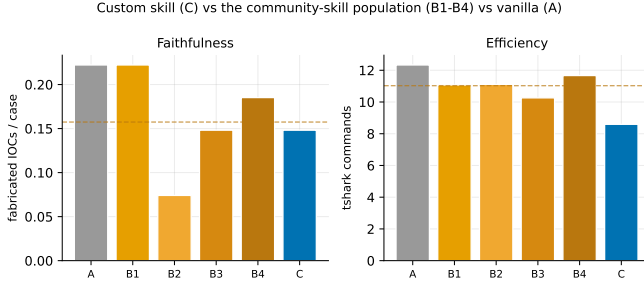


Fig. 8. The custom skill (C) against the four-skill community population (B1–B4) and the untreated baseline (A). C issues the fewest tshark commands of any skill shown, its one significant advantage. Under strict evidence-grounded scoring, fabrication is low and statistically indistinguishable across arms. The dashed line marks the community-population mean, so the comparison rests on a population of widely adopted skills rather than a single baseline.

C. Component ablation

Ablating the components of the custom skill one by one (arms D1–D3) tests what accounts for the tool-call reduction (Fig. 9). In terms of accuracy and faithfulness, the ablations, like the main arms, do not produce any noticeable effects: recall (0.88–0.91), family attribution (0.44–0.52), and fabrication (0.07–0.26) stay within noise level, and none of the

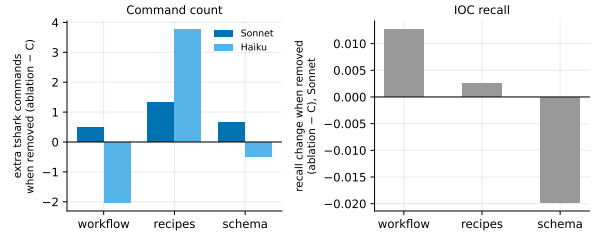


Fig. 9. Effect of removing each component, computed as the ablation minus the full skill (C), case-level means. Left: extra tshark commands on both models; only removing the recipes adds commands on both. Right: change in IOC recall on Sonnet; all three changes are within ± 0.02 and none is significant.

single-component differences stays significant after correction at $n=9$. Ablating the *workflow* (D1) does not reduce recall or attribution on the clean data; both values are nominally slightly higher with the ablation. Hence, we withdraw the more confident interpretation, derived from our previous mixed-epoch data, that the workflow drives accuracy. All ablations agree on the *command count*. The complete skill (C) shows the smallest command usage (8.6), while each ablation generates more commands (no-workflow 9.1, no-schema 9.3, no-recipes

9.9). The exclusion of the `tshark recipes` comes with the largest price: the difference between C and D2 is +1.3 commands and is nominally significant ($p=0.02$ uncorrected) but not after correction. The Haiku replication confirms this pattern (Fig. 9, left). Without the recipes, the agent again issues the largest number of commands (23.8 vs 20.1, +3.8, $p=0.09$), but without the workflow (18.0, $p=0.18$) or the schema (19.6, $p=0.33$), the Haiku agent is *leaner* than with the full skill. Hence, the recipes are the only component of the skill whose ablation increases the command count in both models. The effects of the workflow and schema change direction between models, and we make no claim for them. The output *schema* remains neutral along all dimensions. Omitting it changes neither recall nor fabrication significantly; in fact, D3 fabricates nominally *more* than C (0.26 vs 0.15), further weakening any “the schema pressures fabrication” interpretation. The ablation should be treated as a ranking by directional impact on tool usage, and it leads to one conclusion: invest in concrete command recipes rather than a rigid output template.

D. Attribution is the hard ceiling

The maximum attainable level of family attribution for each arm is 5 of 9 cases. The families missed every time (OsKi Stealer, WarmCookie, BazarLoader, and the fake-authenticator infostealer) coincide with cases where there are no IDS alerts and the family has to be derived from traffic alone. In all cases where Suricata alerts are present, each arm uses them and attributes correctly; without them, no skill bridges the gap. This puts a bound on what can be accomplished with packet-only agentic attribution.

E. Cross-model replication (Haiku)

To check whether the results are due to skills or to a particular model, the full nine-arm protocol was repeated using a smaller model (Claude Haiku), with three repeats per case, resulting in 239 valid runs (Table IV). The reduction in tool calls is confirmed and is the only contrast significant on *both* models: the custom skill cuts `tshark` commands by 35% on Haiku (paired $t=-5.39$, $p<0.001$, $d_z=-1.80$), compared to 30% on Sonnet. The output-token reduction is significant on Haiku (11%, $p=0.027$) but not on Sonnet ($p=0.22$). Wall-clock reductions are not significant on either model ($p=0.09$ on Sonnet, $p=0.31$ on Haiku); per-run wall-clock variability is substantial, and a single slow run can heavily skew the average per case. For both models, the consistent outcome is a reduced *command count*, not a uniform token or latency win.

As far as accuracy goes, the full Haiku dataset erases a trend seen in a single repetition. There is virtually no difference in IOC recall between the vanilla and custom configurations ($p=0.97$), while the improvement in victim ID from 0.88 to 0.96 is not statistically significant ($p=0.08$). The “weak model gains accuracy” trend seen in rep 1 was just noise. Both models arrive at the same conclusion: a skill changes tool-call cost but does not improve correctness.

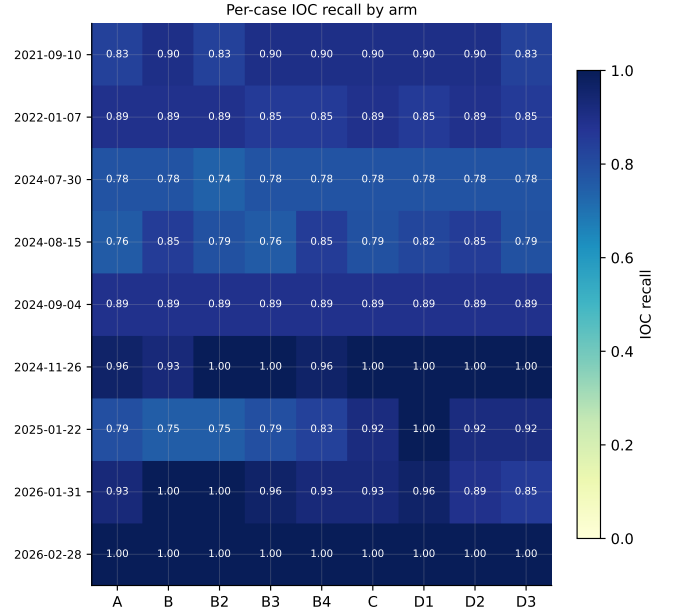


Fig. 10. Per-case IOC recall by arm.

F. Per-case detail and error analysis

Table V gives the complete per-case grid, and Fig. 10 its heatmap representation. Recall stays at ceiling or close to it for most of the captures; the dispersion among arms is due to some hard cases, not to any general shift. Two types of errors are seen repeatedly, and concrete examples of them follow.

Fabrication. On the Lumma capture, one IPv4 address (172.56.88.98) was reported by arm C as a C2 endpoint, although it was never observed in any packet, neither as a frame endpoint nor as a DNS answer. Under a less restrictive universe without DNS answers, the same run scored four fabrications, three of which were legitimate resolved addresses; this is why the universe was completed and the runs re-scored. On this capture, the no-schema arm (D3) left the field empty, but this pattern does not generalize. D3 showed fabrication on two other captures where C does not, meaning that fabrication cannot be considered a schema effect (Section V).

Suppressed exploration (the skill-issue effect). At an early stage, in the presence of the custom skill’s `tshark`-centered recipes, the agent went straight to the capture, omitting the enumeration of the evidence directory and hence the IDS alert file naming the family, while the untreated agent explored and opened the directory. An explicit evidence-inventory step in the workflow counteracted the effect. In the final dataset, every arm, including the no-workflow ablation, opened the IDS alert file in all 6 alert-bearing runs.

VI. DISCUSSION

Given a strong base model, skills serve as efficiency utilities for the agent, not as capability or safety enhancements. The return on a network-forensics skill built on a capable model is therefore a leaner *tool-use footprint*, not an improved

TABLE IV

CROSS-MODEL COMPARISON OF VANILLA (A) AND THE CUSTOM SKILL (C), CASE-LEVEL MEANS (SONNET AND HAIKU, THREE REPS EACH). THE TOOL-CALL REDUCTION IS SIGNIFICANT ON BOTH MODELS. THE TOKEN REDUCTION IS SIGNIFICANT ON HAIKU ONLY, AND THE WALL-CLOCK REDUCTION ON NEITHER; HAIKU A’S WALL-CLOCK IS INFLATED BY A FEW SLOW RUNS ($p=0.31$). ACCURACY DOES NOT DIFFER ON EITHER MODEL.

Model	Arm	Recall	Victim	Halluc.↓	tshark↓	Wall s.↓
Haiku	A	0.776	0.880	0.19	30.8	447
	C	0.778	0.963	0.15	20.1	151
Sonnet	A	0.870	1.000	0.22	12.3	188
	C	0.898	1.000	0.15	8.6	163

TABLE V

PER-CASE IOC RECALL FOR ARMS A, B1, AND C, THE STANDARD DEVIATION OF RECALL ACROSS THOSE ARMS, AND ARM C’S HALLUCINATION AND TSHARK COUNT. MOST CASES SIT NEAR CEILING, AND THE SPREAD IS CONCENTRATED IN A FEW HARD CAPTURES.

Case	IOC recall			Halluc. / tshark (C)		
	A	B1	C	rec. sd	hal. C	tshk C
2021-09-10	0.83	0.90	0.90	0.04	0.0	8
2022-01-07	0.89	0.89	0.89	0.00	0.0	7
2024-07-30	0.78	0.78	0.78	0.00	0.3	8
2024-08-15	0.76	0.85	0.79	0.05	0.0	9
2024-09-04	0.89	0.89	0.89	0.00	0.0	8
2024-11-26	0.96	0.93	1.00	0.04	0.0	7
2025-01-22	0.79	0.75	0.92	0.09	0.7	10
2026-01-31	0.93	1.00	0.93	0.04	0.0	11
2026-02-28	1.00	1.00	1.00	0.00	0.3	9

accuracy ceiling or measurably fewer fabrications. The leaner footprint is a real saving: a 30% reduction in tool calls without any significant recall difference, replicated on two models and significant even after correction for multiple comparisons, matters in a SOC environment where tool invocations are metered. Reductions in token usage and wall-clock time are consistent with this trend but fail to reach significance at $n=9$; no latency advantage can therefore be claimed. Teams expecting a skill to lift a weak model to competence or to make a good model more truthful must measure, not assume.

There are limits to the utility of the output schema. Component-level ablation isolates the areas in which effort expended in skill authoring bears fruit. Removing the strict typed-IOC output schema preserved recall and, with strict evidence-based scoring, a similar fabrication rate; thus, on this task the output schema shows no measurable benefit. One methodological note concerning our approach: with less strict scoring criteria, the schema initially appeared to *increase* fabrication, but the effect disappeared upon adding DNS-answer and payload fields. Faithfulness metrics rely on the correctness of the observability model behind them. Skill authors would find concrete command recipes more useful than an inflexible output schema. Where an output schema is used, the statement “no indicator of this type was observed” must be treated as a first-class answer.

Skill-issue effect. In the pilot failure outlined in Section V, the tshark-centric recipes blocked the agent from inspecting the evidence directory and cost it a family attribution that the unaltered baseline made. The narrow focus of a skill may prevent the exploration typical of a generalist agent. The larger point is that a skill must expand, not limit, the evidentiary

space inspected by an agent.

VII. THREATS TO VALIDITY

All ground-truth data have only one source (MTA); evidence validation per capture and malware-family diversity mitigate this limitation. The skill was incorporated into the experiment as a system preamble rather than through the native skill loader, thereby separating content from retrieval; effects related to the native skill loader will be addressed in future work. The efficiency effect replicates on another model (Haiku); generalization to more vendors will be part of future research. At $n=9$, differences between conditions in terms of accuracy and faithfulness are underpowered compared to case-level variance (recall sd ≈ 0.08 ; fabrication has low values, sd 0.15–0.36); therefore, they are reported as nulls rather than claims. The one effect that is large, survives multiple-testing correction, and holds across models is the tool-call reduction. We collected three repetitions per condition across all nine arms and two models (482 valid runs: 243 Sonnet, 239 Haiku). For the primary model, all nine arms were re-collected in a *single continuous batch*, with three repetitions for each of the 9 cases and 9 arms. In this way, we eliminate the collection-epoch confound that existed in a previous version of this dataset collected in mixed batches, because all cross-arm contrasts now fall within one epoch. Data for the secondary model (Haiku) were collected in one batch as well. The model is an unpinning alias, which is a reproducibility limitation we acknowledge. However, each batch spanned no more than a few days; thus residual build drift is small but not provably zero, and exact bit-level replication would require exact build pinning.

The custom skill itself is a further limitation. Its workflow was refined after a failure on one evaluation capture; hence arm C was not developed on a held-out set. The ablation contrasts (C versus its constituent components) are internal to the skill and are not affected by this, and the efficiency effect replicates on another model. Still, the standing of C relative to the community skills could be somewhat optimistic for this reason; a blind, held-out evaluation of a frozen skill will be future work.

VIII. CONCLUSION

We divide the deployment of procedural knowledge for LLM-agent network forensics into a nine-arm experiment. In the case of a good base model, a custom skill improves tool-use efficiency but not accuracy or faithfulness. Our method reaches identical conclusions through the use of far fewer commands (30% reduction when compared to using no skill at all, $p < 0.001$, replicated on a second model), while not compromising recall and with fabrication indistinguishable across arms. In the component ablation, concrete command recipes emerge as the only component whose absence results in more commands for both models, while the influence of workflow and schema is inconsistent across models. Command recipes should be prioritized by skill developers over output templates, and skills should be created in such a way as to expand the scope of evidence used by the agent. The harness, the evidence-validated dataset, and all skill variants¹ are released for replication and extension.

REFERENCES

- [1] S. Fumero, K. Huang, M. Boffa, D. Giordano, M. Mellia, and D. Rossi, "CyberSleuth: Autonomous blue-team LLM agent for web attack forensics," *arXiv preprint arXiv:2508.20643*, 2025.
- [2] H. Chen, Z. Zhang, J. Jiang, Q. Su, and Q. Xiang, "Holmes: An evidence-grounded LLM agent for auditable DDoS investigation in cloud networks," 2026, arXiv:2601.14601.
- [3] S. Jajodia, M. Sultana, S. Majumdar, A. Taylor, and G. Vandenbergh, "Before you hand over the wheel: Evaluating LLMs for security incident analysis," *arXiv preprint arXiv:2603.06422*, 2026.
- [4] Anthropic, "Introducing agent skills," 2025, <https://www.anthropic.com/news/skills>.
- [5] Agent Skills, "Agent skills specification," 2026, <https://agentskills.io>, accessed 2026-10-01.
- [6] X. Li *et al.*, "SkillsBench: Benchmarking how well agent skills work across diverse tasks," 2026, arXiv:2602.12670.
- [7] S. J. Chacko, J. Huggelstone, C. M. Islam, and X. Liu, "When skills don't help: A negative result on procedural knowledge for tool-grounded agents in offensive cybersecurity," *arXiv preprint arXiv:2605.20023*, 2026.
- [8] W.-J. Huang, "Do LLM-generated skills make better AI data scientists? a component ablation across data-science workflows," *arXiv preprint arXiv:2607.07504*, 2026.
- [9] Y. Wang, Y. Zhou, Y. Liang, C. Zhang, F. Liu, J. Zhou, and H. Yao, "Not all skills help: Measuring and repairing agent knowledge," *arXiv preprint arXiv:2606.15390*, 2026.
- [10] P. Kassianik, B. Nelson, and Y. Singer, "Beyond success rate: Cost-aware evaluation of offensive and defensive security agents," *arXiv preprint arXiv:2607.15263*, 2026.
- [11] T. Zhang and Z. Qi, "Skill-to-LoRA: From using skills to learning behaviors for token-efficient LLM agents," *arXiv preprint arXiv:2606.16769*, 2026.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [14] L. Ouyang, J. Wu, X. Jiang, D. Almeida *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [17] J. Wei, Y. Tay, R. Bommasani, C. Raffel *et al.*, "Emergent abilities of large language models," *Transactions on Machine Learning Research (TMLR)*, 2022.
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [19] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [20] Y. Qin, S. Liang, Y. Ye, K. Zhu *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," in *International Conference on Learning Representations (ICLR)*, 2024.
- [21] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [22] W. Kim, S. Park, Y. In, S. Kim, D. Lee *et al.*, "Beyond the final answer: Evaluating the reasoning trajectories of tool-augmented agents," *arXiv preprint arXiv:2510.02837*, 2025.
- [23] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
- [24] H. Xu, C. Li, X. Ma, X. Ou, Z. Zhang *et al.*, "The evolution of tool use in LLM agents: From single-tool call to multi-tool orchestration," *arXiv preprint arXiv:2603.22862*, 2026.
- [25] Anthropic, "Equipping agents for the real world with agent skills," 2025, <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- [26] G. Bakal, "Knowledge activation: AI skills as the institutional knowledge primitive for agentic software development," *arXiv preprint arXiv:2603.14805*, 2026.
- [27] F. Li, A. Zhang, and C. Lv, "Constraint tax in open-weight LLMs: An empirical study of tool calling suppression under structured output constraints," *arXiv preprint arXiv:2606.25605*, 2026.
- [28] A. Shahriar, M. N. Rahman, S. Ahmed, F. Sadeque, and M. R. Parvez, "A survey on agentic security: Applications, threats and defenses," *arXiv preprint arXiv:2510.06445*, 2025.
- [29] Y. Xu, Y. Zhuang, X. Liu, T. Zhang, B. Xiao *et al.*, "LLM agents security duality: A comprehensive survey of self-security and empowered cybersecurity," *arXiv preprint arXiv:2606.28450*, 2026.
- [30] Z. He, J. Dong, Z. Li, T. Chen, and G. Deng, "A survey of LLM-driven penetration testing: Taxonomy, co-evolution, and open challenges," *arXiv preprint arXiv:2607.02605*, 2026.
- [31] J. Peng, Z. Li, C. You, Y. Wang, and H. Sun, "Hackers or hallucinators? a comprehensive analysis of LLM-based automated penetration testing," *arXiv preprint arXiv:2604.05719*, 2026.
- [32] M. Kobayashi, M. Fuchi, A. Zanashir, T. Yoneda, and T. Takagi, "Construction and evaluation of LLM-based agents for semi-autonomous penetration testing," *arXiv preprint arXiv:2502.15506*, 2025.
- [33] M. Shao, H. Xi, N. Rani, M. Udeshi, and V. S. C. Putrevu, "CRAKEN: Cybersecurity LLM agent with knowledge-based execution," *arXiv preprint arXiv:2505.17107*, 2025.
- [34] A. Borah, M. T. Alam, and N. Rastogi, "Adapting large language models to emerging cybersecurity using retrieval-augmented generation," *arXiv preprint arXiv:2510.27080*, 2025.

¹<https://github.com/yaakulya123/skill-issue-pcap-forensics>

- [35] S. Ndichu, T. Ban, S. Ozawa, T. Takahashi, and D. Inoue, "AI-driven security alert screening and alert fatigue mitigation in security operations centers: A survey," *arXiv preprint arXiv:2605.08316*, 2026.
- [36] S. Jang, M. Lee, and T. Shon, "Local LLM-based cyber incident analysis in air-gapped networks via teacher-student knowledge distillation and agentic orchestration," *Electronics*, vol. 15, no. 13, p. 2949, 2026.
- [37] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan *et al.*, "Purple llama CyberSecEval: A secure coding benchmark for language models," *arXiv preprint arXiv:2312.04724*, 2023.
- [38] L. Deason *et al.*, "CyberSOCEval: Benchmarking LLMs capabilities for malware analysis and threat intelligence reasoning," *arXiv preprint arXiv:2509.20166*, 2025.
- [39] T. Cui, X. Lin, S. Li, M. Chen, Q. Yin, Q. Li, and K. Xu, "TrafficLLM: Enhancing large language models for network traffic analysis with generic traffic representation," *arXiv preprint arXiv:2504.04222*, 2025.
- [40] J. Li, Y. Chai, L. Du, C. Duan, H. Yan, and Z. Gu, "GRIDAI: Generating and repairing intrusion detection rules via collaboration among multiple LLM-based agents," 2025, arXiv:2510.13257.
- [41] D. A. Worae and S. Mastorakis, "An LLM-powered AI agent framework for holistic IoT traffic interpretation," 2025, arXiv:2510.13925.
- [42] Open Information Security Foundation, "Suricata: Open source intrusion detection and prevention engine," 2024, <https://suricata.io>.
- [43] The Zeek Project, "Zeek network security monitor," 2024, <https://zeek.org>.
- [44] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization (CIC-IDS2017)," 2018, iCISSP.
- [45] R. Jindal and A. Anwar, "Emerging trends of recently published datasets for intrusion detection systems: A survey," *arXiv preprint arXiv:2110.00773*, 2021.
- [46] Z. Xu and Y. Liu, "Robust anomaly detection in network traffic: Evaluating machine learning models on CICIDS2017," *arXiv preprint arXiv:2506.19877*, 2025.
- [47] M. A. Talukder, M. M. Islam, M. A. Uddin, K. F. Hasan, and S. Sharmin, "Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction," *arXiv preprint arXiv:2401.12262*, 2024.
- [48] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep., 2018.
- [49] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, 2023.
- [50] L. Huang, W. Yu, W. Ma, W. Zhong *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, 2025.
- [51] P. Lewis, E. Perez, A. Piktus, F. Petroni *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [52] B. Duncan, "Malware-traffic-analysis.net: Traffic analysis exercises," 2024, <https://www.malware-traffic-analysis.net>.